
Sample Tables Used

Assume these tables:

```
1  Employees
2  -----
3  emp_id
4  emp_name
5  department_id
6  salary
7  hire_date
8  manager_id
9
10 Departments
11 -----
12 department_id
13 department_name
14
15 Orders
16 -----
17 order_id
18 customer_id
19 order_date
20 amount
21
22 Customers
23 -----
24 customer_id
25 customer_name
26 city
```

1. Basic SQL Interview Queries

1. Fetch all records from a table

Question: Write a query to display all employees.

```
1  SELECT *
2  FROM Employees;
```

2. Select specific columns

Question: Display employee name and salary.

```
1 SELECT emp_name, salary
2 FROM Employees;
```

3. Use WHERE condition

Question: Find employees whose salary is greater than 50,000.

```
1 SELECT *
2 FROM Employees
3 WHERE salary > 50000;
```

4. Use AND condition

Question: Find employees from department 10 whose salary is greater than 50,000.

```
1 SELECT *
2 FROM Employees
3 WHERE department_id = 10
4 AND salary > 50000;
```

5. Use OR condition

Question: Find employees from department 10 or department 20.

```
1 SELECT *
2 FROM Employees
3 WHERE department_id = 10
4 OR department_id = 20;
```

6. Use IN

Question: Find employees from departments 10, 20, and 30.

```
1 SELECT *
2 FROM Employees
3 WHERE department_id IN (10, 20, 30);
```

7. Use BETWEEN

Question: Find employees whose salary is between 40,000 and 80,000.

```
1 SELECT *  
2 FROM Employees  
3 WHERE salary BETWEEN 40000 AND 80000;
```

8. Use LIKE

Question: Find employees whose name starts with A.

```
1 SELECT *  
2 FROM Employees  
3 WHERE emp_name LIKE 'A%';
```

9. Find names ending with a letter

Question: Find employees whose name ends with n.

```
1 SELECT *  
2 FROM Employees  
3 WHERE emp_name LIKE '%n';
```

10. Use ORDER BY

Question: Display employees sorted by salary in descending order.

```
1 SELECT *  
2 FROM Employees  
3 ORDER BY salary DESC;
```

2. Aggregate Function Queries

11. Count total employees

Question: Find the total number of employees.

```
1 SELECT COUNT(*) AS total_employees  
2 FROM Employees;
```

12. Find maximum salary

Question: Find the highest salary.

```
1 SELECT MAX(salary) AS highest_salary
2 FROM Employees;
```

13. Find minimum salary

Question: Find the lowest salary.

```
1 SELECT MIN(salary) AS lowest_salary
2 FROM Employees;
```

14. Find average salary

Question: Find the average salary of employees.

```
1 SELECT AVG(salary) AS average_salary
2 FROM Employees;
```

15. Find total salary

Question: Find the total salary paid to all employees.

```
1 SELECT SUM(salary) AS total_salary
2 FROM Employees;
```

3. GROUP BY and HAVING Queries

16. Count employees in each department

Question: Find the number of employees in each department.

```
1 SELECT department_id, COUNT(*) AS employee_count
2 FROM Employees
3 GROUP BY department_id;
```

17. Average salary by department

Question: Find average salary department-wise.

```
1 SELECT department_id, AVG(salary) AS average_salary
2 FROM Employees
3 GROUP BY department_id;
```

18. Departments having more than 5 employees

Question: Find departments where employee count is greater than 5.

```
1 SELECT department_id, COUNT(*) AS employee_count
2 FROM Employees
3 GROUP BY department_id
4 HAVING COUNT(*) > 5;
```

19. Departments with average salary greater than 60,000

```
1 SELECT department_id, AVG(salary) AS average_salary
2 FROM Employees
3 GROUP BY department_id
4 HAVING AVG(salary) > 60000;
```

4. Joins Interview Queries

20. INNER JOIN

Question: Display employee name with department name.

```
1 SELECT e.emp_name, d.department_name
2 FROM Employees e
3 INNER JOIN Departments d
4     ON e.department_id = d.department_id;
```

21. LEFT JOIN

Question: Display all employees and their department names, including employees without a department.

```
1 SELECT e.emp_name, d.department_name
2 FROM Employees e
3 LEFT JOIN Departments d
4     ON e.department_id = d.department_id;
```

22. RIGHT JOIN

Question: Display all departments and employees working in them.

```
1 SELECT e.emp_name, d.department_name
2 FROM Employees e
3 RIGHT JOIN Departments d
4     ON e.department_id = d.department_id;
```

23. Find employees without department

```
1 SELECT e.*
2 FROM Employees e
3 LEFT JOIN Departments d
4     ON e.department_id = d.department_id
5 WHERE d.department_id IS NULL;
```

24. Find departments with no employees

```
1 SELECT d.*
2 FROM Departments d
3 LEFT JOIN Employees e
4     ON d.department_id = e.department_id
5 WHERE e.emp_id IS NULL;
```

5. DISTINCT and Duplicate Queries

25. Find unique departments from Employees table

```
1 SELECT DISTINCT department_id
2 FROM Employees;
```

26. Find duplicate employee names

```
1 SELECT emp_name, COUNT(*) AS duplicate_count
2 FROM Employees
3 GROUP BY emp_name
4 HAVING COUNT(*) > 1;
```

27. Find duplicate records based on name and department

```
1 SELECT emp_name, department_id, COUNT(*) AS duplicate_count
2 FROM Employees
3 GROUP BY emp_name, department_id
4 HAVING COUNT(*) > 1;
```

6. Subquery Interview Questions

28. Find employees earning more than average salary

```
1 SELECT *
2 FROM Employees
3 WHERE salary > (
4     SELECT AVG(salary)
5     FROM Employees
6 );
```

29. Find employee with highest salary

```
1 SELECT *
2 FROM Employees
3 WHERE salary = (
4     SELECT MAX(salary)
5     FROM Employees
6 );
```

30. Find employee with second highest salary

```
1 SELECT MAX(salary) AS second_highest_salary
2 FROM Employees
3 WHERE salary < (
4     SELECT MAX(salary)
5     FROM Employees
6 );
```

31. Find employees working in the same department as employee John

```
1  SELECT *
2  FROM Employees
3  WHERE department_id = (
4      SELECT department_id
5      FROM Employees
6      WHERE emp_name = 'John'
7  );
```

7. Intermediate SQL Queries

32. Second highest salary using DENSE_RANK

```
1  SELECT emp_id, emp_name, salary
2  FROM (
3      SELECT emp_id,
4             emp_name,
5             salary,
6             DENSE_RANK() OVER (ORDER BY salary DESC) AS
salary_rank
7      FROM Employees
8  ) ranked_employees
9  WHERE salary_rank = 2;
```

33. Nth highest salary

Question: Find the 3rd highest salary.

```
1  SELECT emp_id, emp_name, salary
2  FROM (
3      SELECT emp_id,
4             emp_name,
5             salary,
6             DENSE_RANK() OVER (ORDER BY salary DESC) AS
salary_rank
7      FROM Employees
8  ) ranked_employees
9  WHERE salary_rank = 3;
```

34. Highest salary in each department

```
1 SELECT emp_id, emp_name, department_id, salary
2 FROM (
3     SELECT emp_id,
4            emp_name,
5            department_id,
6            salary,
7            DENSE_RANK() OVER (
8                PARTITION BY department_id
9                ORDER BY salary DESC
10           ) AS salary_rank
11     FROM Employees
12 ) ranked_employees
13 WHERE salary_rank = 1;
```

35. Top 3 salaries in each department

```
1 SELECT emp_id, emp_name, department_id, salary
2 FROM (
3     SELECT emp_id,
4            emp_name,
5            department_id,
6            salary,
7            DENSE_RANK() OVER (
8                PARTITION BY department_id
9                ORDER BY salary DESC
10           ) AS salary_rank
11     FROM Employees
12 ) ranked_employees
13 WHERE salary_rank <= 3;
```

36. Difference between ROW_NUMBER, RANK, and DENSE_RANK

Question: Rank employees based on salary.

```
1 SELECT emp_name,
2        salary,
3        ROW_NUMBER() OVER (ORDER BY salary DESC) AS row_num,
```

```
4      RANK() OVER (ORDER BY salary DESC) AS rank_num,  
5      DENSE_RANK() OVER (ORDER BY salary DESC) AS  
dense_rank_num  
6  FROM Employees;
```

Explanation:

- ROW_NUMBER() gives a unique sequence number.
 - RANK() skips ranking after duplicates.
 - DENSE_RANK() does not skip ranking after duplicates.
-

8. Date-Based SQL Queries

37. Employees hired in the current year

For SQL Server:

```
1  SELECT *  
2  FROM Employees  
3  WHERE YEAR(hire_date) = YEAR(GETDATE());
```

For MySQL:

```
1  SELECT *  
2  FROM Employees  
3  WHERE YEAR(hire_date) = YEAR(CURDATE());
```

38. Employees hired in the last 30 days

For SQL Server:

```
1  SELECT *  
2  FROM Employees  
3  WHERE hire_date >= DATEADD(DAY, -30, GETDATE());
```

For MySQL:

```
1  SELECT *  
2  FROM Employees  
3  WHERE hire_date >= DATE_SUB(CURDATE(), INTERVAL 30 DAY);
```

39. Monthly order amount

```
1  SELECT
```

```
2     YEAR(order_date) AS order_year,  
3     MONTH(order_date) AS order_month,  
4     SUM(amount) AS total_amount  
5 FROM Orders  
6 GROUP BY YEAR(order_date), MONTH(order_date)  
7 ORDER BY order_year, order_month;
```

9. NULL Handling Queries

40. Find employees without manager

```
1 SELECT *  
2 FROM Employees  
3 WHERE manager_id IS NULL;
```

41. Replace NULL department with text

For SQL Server:

```
1 SELECT emp_name,  
2         ISNULL(CAST(department_id AS VARCHAR), 'No Department')  
AS department_status  
3 FROM Employees;
```

For MySQL:

```
1 SELECT emp_name,  
2         IFNULL(department_id, 'No Department') AS  
department_status  
3 FROM Employees;
```

42. Use COALESCE

```
1 SELECT emp_name,  
2         COALESCE(manager_id, 0) AS manager_id  
3 FROM Employees;
```

10. CASE Statement Queries

43. Categorize employees based on salary

```
1  SELECT emp_name,  
2         salary,  
3         CASE  
4             WHEN salary >= 80000 THEN 'High Salary'  
5             WHEN salary >= 50000 THEN 'Medium Salary'  
6             ELSE 'Low Salary'  
7         END AS salary_category  
8  FROM Employees;
```

44. Count employees by salary category

```
1  SELECT  
2      CASE  
3          WHEN salary >= 80000 THEN 'High Salary'  
4          WHEN salary >= 50000 THEN 'Medium Salary'  
5          ELSE 'Low Salary'  
6      END AS salary_category,  
7      COUNT(*) AS employee_count  
8  FROM Employees  
9  GROUP BY  
10     CASE  
11         WHEN salary >= 80000 THEN 'High Salary'  
12         WHEN salary >= 50000 THEN 'Medium Salary'  
13         ELSE 'Low Salary'  
14     END;
```

11. Self Join Queries

45. Employee and manager name

Question: Display employee name along with manager name.

```
1  SELECT e.emp_name AS employee_name,  
2         m.emp_name AS manager_name  
3  FROM Employees e  
4  LEFT JOIN Employees m
```

```
5      ON e.manager_id = m.emp_id;
```

46. Employees having the same manager

```
1  SELECT e1.emp_name AS employee_1,  
2         e2.emp_name AS employee_2,  
3         e1.manager_id  
4  FROM Employees e1  
5  INNER JOIN Employees e2  
6      ON e1.manager_id = e2.manager_id  
7      AND e1.emp_id < e2.emp_id;
```

12. Common Order/Customer Queries

47. Total amount spent by each customer

```
1  SELECT c.customer_id,  
2         c.customer_name,  
3         SUM(o.amount) AS total_spent  
4  FROM Customers c  
5  INNER JOIN Orders o  
6      ON c.customer_id = o.customer_id  
7  GROUP BY c.customer_id, c.customer_name;
```

48. Customers who have not placed any orders

```
1  SELECT c.*  
2  FROM Customers c  
3  LEFT JOIN Orders o  
4      ON c.customer_id = o.customer_id  
5  WHERE o.order_id IS NULL;
```

49. Customer with highest total spending

```
1  SELECT customer_id, total_spent  
2  FROM (  
3      SELECT customer_id,  
4             SUM(amount) AS total_spent,
```

```

5         DENSE_RANK() OVER (ORDER BY SUM(amount) DESC) AS
spending_rank
6     FROM Orders
7     GROUP BY customer_id
8 ) ranked_customers
9 WHERE spending_rank = 1;

```

50. Latest order of each customer

```

1 SELECT customer_id, order_id, order_date, amount
2 FROM (
3     SELECT customer_id,
4            order_id,
5            order_date,
6            amount,
7            ROW_NUMBER() OVER (
8                PARTITION BY customer_id
9                ORDER BY order_date DESC
10           ) AS rn
11     FROM Orders
12 ) latest_orders
13 WHERE rn = 1;

```

13. Frequently Asked Theory Questions

51. Difference between WHERE and HAVING

WHERE	HAVING
Filters rows before grouping	Filters groups after grouping
Used before GROUP BY	Used after GROUP BY
Cannot use aggregate functions directly	Can use aggregate functions

Example:

```

1 SELECT department_id, COUNT(*) AS employee_count
2 FROM Employees
3 WHERE salary > 50000
4 GROUP BY department_id
5 HAVING COUNT(*) > 3;

```

52. Difference between DELETE, TRUNCATE, and DROP

Command	Meaning
DELETE	Deletes selected rows
TRUNCATE	Deletes all rows quickly
DROP	Deletes the table structure completely

53. Difference between INNER JOIN and LEFT JOIN

INNER JOIN	LEFT JOIN
Returns matching records only	Returns all records from left table
Non-matching records are removed	Non-matching left table records appear with NULL

54. Difference between UNION and UNION ALL

UNION	UNION ALL
Removes duplicates	Keeps duplicates
Slower	Faster
Performs distinct operation	Does not perform distinct operation

Example:

1	SELECT city FROM Customers
2	UNION
3	SELECT city FROM Suppliers;
1	SELECT city FROM Customers
2	UNION ALL
3	SELECT city FROM Suppliers;

14. Most Important Practice Questions

Try these yourself:

1. Find the third highest salary.
2. Find duplicate employee records.
3. Find employees earning more than department average salary.
4. Find top 2 employees from each department based on salary.
5. Find customers who never placed an order.
6. Find monthly sales amount.
7. Find department-wise highest salary.
8. Find employees hired in the last 6 months.
9. Find employee and manager names.
10. Find the latest order for each customer.

Quick Revision Topics for SQL Interview

Focus on these topics:

- SELECT, WHERE, ORDER BY
- GROUP BY, HAVING
- Aggregate functions: COUNT, SUM, AVG, MIN, MAX
- Joins: INNER, LEFT, RIGHT, FULL
- Subqueries
- Window functions
- ROW_NUMBER, RANK, DENSE_RANK
- CASE
- NULL handling
- Date functions
- Duplicate records
- Second highest salary
- Department-wise top salary

If you want, I can also give you a **SQL interview practice set with sample data and answers** or a **PDF-style cheat sheet**.